Laravel After Deploy

Architecture, Performance, and Operations at Scale

Nuruzzaman Milon

Laravel After Deploy

Architecture, Performance, and Operations at Scale

Nuruzzaman Milon

First published: August 2026

Written with	Markdown. Parts were dictated with Hex, an open-source, on-device voice-transcription tool. The rest was typed.
Produced with	Ibis Next, using mPDF for PDF generation and CommonMark for HTML rendering.
Trim size	7.44in x 9.69in (Crown Quarto)
Typefaces	Linux Libertine (body text), Times (headings), and 0xProto (code, SIL Open Font License 1.1).
Code width	66 characters per line, so a listing never wraps awkwardly on the page.

To my family, for making every effort worthwhile.

Preface

Shipping a Laravel application is the easy part. The hard part starts after deploy. A flash sale or viral post can turn normal traffic into a spike. Bots can flood your login endpoint. A dependency you don't control can fail, and the same webhook can arrive twice. Then the on-call phone rings at 3 a.m., and the framework documentation does not contain the answer.

This book is about that layer. It covers the decisions that keep a real production system fast, correct, and operable.

Who this book is for

This book is for a mid-to-senior Laravel engineer who already knows how to build a Laravel application and is now responsible for keeping one alive. A growing team gives this engineer responsibility for architecture decisions, performance incidents, and migrations that must happen without a maintenance window.

I assume you already know how routing, Eloquent, queues, and testing work in Laravel. I do not stop to explain the framework. Instead, every page addresses the decisions above the framework:

- How do you structure a codebase when several teams work in it?
- How do you keep the system fast when traffic no longer fits comfortably on one database?
- How do you keep the system correct when the same request can arrive twice?
- How do you keep it available when a dependency you don't control fails?
- How do you operate all of that during a 3 a.m. incident without creating a long-term problem?

This is not a Laravel tutorial about defining routes, writing migrations, or using Eloquent relationships. Excellent books already cover those subjects. If you now face a monorepo, a queue backlog, or a balance that does not add up, keep reading.

How to read it

The book has eight parts, ordered by dependency rather than importance. Reading from front to back gives you the clearest path, but the book is also designed to work as a production reference.

Parts 1 and 2 introduce vocabulary and patterns used later. Part 5 is also worth reading early because its observability terms appear throughout the deployment and operations chapters.

If you are dealing with an active incident, jump directly to the relevant chapter. You do not need to finish the architecture chapters before reading about a queue backlog, database failover, or broken CI pipeline. The chapter recaps can help you find the main idea quickly.

Read Chapter 38 last. The capstone follows one system through a serious incident and shows how decisions from the earlier parts interact.

Each chapter begins with an anonymized production failure, explains the pattern that could have prevented or fixed it, and ends with a short recap. These incidents come from more than fifteen years of work across several languages and frameworks. When an original system did not use Laravel, I translated the failure into a Laravel example while preserving the underlying lesson.

About the examples

Every incident in this book happened in a real production system. The failure modes, trade-offs, and fixes come from systems I helped build and operate. They are not thought experiments.

The identifying details are fictional. I changed, generalized, or combined company names, product names, team names, exact numbers, and any other details that could identify an employer or client.

Several fictional businesses appear throughout the book. They include Karbar (an order-processing platform), Hishab (a wallet service), and Drishti (a livestreaming and short-video app). These businesses represent common types of systems. They are not case studies of specific companies, and they do not map directly to any company.

Nothing here discloses a trade secret, proprietary architecture, or confidential information from any organization where I have worked. The anonymization protects those organizations while preserving each lesson. The architecture decisions and opinions in these pages are my own.

The code examples use Laravel because it is this book's main stack. However, most of the ideas are not specific to Laravel. Idempotency, backpressure, observability, failure isolation, and the other topics are production concerns in any language or framework. The patterns still apply if you do not use Laravel, even when the exact code examples do not.

A note on unfamiliar terms

Each chapter spells out abbreviations on first use. If you read chapters out of order, Appendix A (Glossary) collects recurring abbreviations such as SLO, SLA, SLI, MTTR, RPO/RTO, and ADR. Each glossary entry points to the chapter that explains the term.

Part 1 - Foundations: Structuring Laravel Systems

This part establishes the vocabulary and decision frameworks used throughout the book. It begins with monoliths, modular monoliths, and microservices as runtime shapes. It then treats monorepos, shared kernels, service extractions, vendor boundaries, and outbound API clients as separate decisions that teams often confuse with runtime shape.

- Adding a nullable column with no default is metadata-only and near-instant on both MySQL (InnoDB, modern versions) and PostgreSQL, regardless of table size.
- Adding a column with a default value, or adding a `NOT NULL` constraint to an existing column, historically required rewriting every row to populate or validate the new value - a cost proportional to table size. Recent versions of both engines have narrowed this (PostgreSQL avoids a full rewrite for constant defaults since version 11; MySQL's behavior depends on the specific `ALGORITHM` used), but the safe assumption is: verify the specific operation against the specific engine version in use, on a copy of production-sized data, before trusting it in staging.
- Renaming a column, dropping a column, or changing a column's type can each range from instant metadata changes to full table rewrites, again depending on engine and version.

The general rule is simple. Do not trust migration timing from a small staging database. Run the migration against a production-sized copy before release. A recent snapshot restored to a disposable environment is suitable. Confirm in advance whether the operation takes a blocking lock.

Concretely, adding a `NOT NULL` constraint to `orders.customer_email` on PostgreSQL as a single step forces the database to scan and validate every one of two hundred million rows while holding a lock that blocks writes for the duration:

```
/*
 * Locks orders for the full validation scan - the whole table,
 * all at once.
 */
Schema::table('orders', function (Blueprint $table) {
    $table->string('customer_email')->nullable(false)->change();
});
```

Splitting it into "add the constraint without enforcing it yet," "validate it separately," and only then "make the column formally `NOT NULL`" turns one long exclusive lock into a near-instant metadata change plus a scan that only takes a brief lock at the very end:

```
-- Step 1: instant - records the constraint without checking
existing rows.
ALTER TABLE orders ADD CONSTRAINT customer_email_not_null
    CHECK (customer_email IS NOT NULL) NOT VALID;

-- Step 2: scans the table, but only takes a brief lock to flip
the
-- constraint's state once the scan finds no violations - not for
the
-- duration of the scan itself.
ALTER TABLE orders VALIDATE CONSTRAINT customer_email_not_null;

-- Step 3: after the validated check proves there are no nulls,
PostgreSQL
-- can set the column attribute without repeating the full-table
scan.
ALTER TABLE orders
    ALTER COLUMN customer_email SET NOT NULL;
```

Run as separate deploys, the first one is safe to ship immediately; validation and the final `SET NOT NULL` can run once the backfill from the next section has finished, with no code depending on their timing.

Concurrent and non-blocking index creation

Adding an index to a large table is one of the most common migrations to get wrong, because a plain `CREATE INDEX` takes a lock that blocks writes for as long as the index build takes - which, on a two-hundred-million-row table, can be tens of minutes.

PostgreSQL's answer is `CREATE INDEX CONCURRENTLY`, which builds the

index without holding a write lock, at the cost of a slower two-pass build and a real failure mode: if it's interrupted, it can leave behind an invalid index that has to be dropped and retried, not resumed.

```
CREATE INDEX CONCURRENTLY idx_orders_customer_created
    ON orders (customer_id, created_at);
```

MySQL's InnoDB engine supports online DDL for many index operations via `ALGORITHM=INPLACE, LOCK=NONE`, which allows concurrent reads and writes during the build - but not every DDL operation supports every algorithm, and the safe habit is to check the specific operation against the running engine version rather than assume.

```
ALTER TABLE orders
    ADD INDEX idx_orders_customer_created (customer_id,
created_at),
    ALGORITHM=INPLACE, LOCK=NONE;
```

Laravel's migration files don't manage this distinction automatically - a `Schema::table()` migration using `$table->index()` issues a plain `CREATE INDEX`/`ADD INDEX` unless the raw SQL or a database-specific driver option is used to request the non-blocking variant. On a large table, that's a decision to make explicitly, not a default to trust:

```
public function up(): void
{
    /*
     * Schema::table()->index() takes a blocking lock for the
     * build - fine on a small table, a production incident on a
     * two-hundred-million-row one.
     */
    DB::statement(
        'CREATE INDEX CONCURRENTLY idx_orders_customer_created'
        .' ON orders (customer_id, created_at)'
    );
}
```

`CREATE INDEX CONCURRENTLY` can't run inside the transaction Laravel wraps each migration in by default, so the migration class also needs `public $withinTransaction = false;` - forgetting that is the most common way this pattern fails silently in a migration file, even though it works fine when run by hand. And because an interrupted concurrent build leaves an unusable index behind rather than rolling back, always check for one before retrying:

```
-- Finds indexes left in an invalid state by an interrupted
CONCURRENTLY build.
SELECT indexrelid::regclass AS index_name
FROM pg_index
WHERE indisvalid = false;

-- Safe to drop and retry - an invalid index is never used by the
planner.
DROP INDEX CONCURRENTLY idx_orders_customer_created;
```

The backward-compatible column rename pattern

On most engines, a direct column rename is a fast metadata operation. The danger is not usually the `ALTER` operation. When the rename commits, every consumer that still uses the old name breaks at once. This includes code outside your control.

Do not rename a column in one step. Use phases that you can deploy, verify, and roll back independently:

Phase	Schema state	Application state	Rollback if something's wrong
1. Add	New column exists, nullable	Not yet used	Drop the new column - no impact

Phase	Schema state	Application state	Rollback if something's wrong
2. Dual-write	Both columns exist	App writes to both columns on every write path; reads still from the old column	Stop dual-writing - old column is still authoritative
3. Backfill	Both columns exist	Batch job populates the new column for existing rows	Backfill is idempotent - safe to re-run or pause
4. Migrate reads	Both columns exist	App reads from the new column; old column no longer read	Revert the read change - old column is still fully populated
5. Stop dual-write	Both columns exist	App writes only to the new column	Revert to dual-writing if any consumer still expects the old column
6. Drop	Old column removed	Fully migrated	Not reversible past this point - this phase only ships once nothing reads the old column

Each phase is a separate, independently deployable change. If phase 4 reveals a bug (the new column has a subtly different meaning than assumed), the rollback is "revert one deploy," not "recover from a completed rename." This is the same expand-and-contract shape used for the shared-package upgrades in Chapter 7 - the pattern generalizes to any change where "old" and "new" need to coexist for a transition window.

Phase 1 is the only phase that touches the database schema directly - it's a plain, additive migration:

```
public function up(): void
{
    Schema::table('orders', function (Blueprint $table) {
        // after() is MySQL-only; omit it on PostgreSQL.
        $table->string(
            'customer_email'
        )->nullable()->after('buyer_email');
    });
}
```

Phase 2's dual-write lives in the model, not in a migration, and is the one line of application code every write path now depends on until phase 5 removes it - with the caveat that "every write path" means every write path that goes through a model instance. `Order::query()->update( ... )` and anything built on `DB::table('orders')` skip model events entirely, so grep for those before trusting this hook, because phase 4's correctness rests on it:

```
protected static function booted(): void
{
    static::saving(function (Order $order) {
        if ($order->isDirty('buyer_email')) {
            $order->customer_email = $order->buyer_email;
        }
    });
}
```

Phase 6's drop is deliberately its own migration, shipped only after the checklist at the end of this chapter is fully checked off:

```
public function up(): void
{
    Schema::table('orders', function (Blueprint $table) {
        $table->dropColumn('buyer_email');
    });
}
```

Batching large backfills

Phase 3 above - populating the new column for two hundred million existing rows - is itself a hazard if done as one statement:

```
// Don't do this on a large table.
DB::table('orders')
    ->update(['customer_email' => DB::raw('buyer_email')]);
```

A single `UPDATE` that touches every row holds many row locks. Depending

on the engine and isolation level, the locks can remain for one long transaction. The update also competes with live traffic. On replicated systems, the large write can create significant replication lag.

Use chunks instead. Update a bounded number of rows in each statement. Pause briefly between batches so replicas can catch up and live traffic can continue.

```
Order::query()
    ->whereNull('customer_email')
    ->orderBy('id')
    ->chunkById(1000, function ($orders) {
        Order::query()
            ->whereIn('id', $orders->pluck('id'))
            ->update(
                ['customer_email' => DB::raw('buyer_email')]
            );

        // 100ms - bound replication lag and lock contention
        usleep(100_000);
    });
```

Running this as a resumable Artisan command that processes one bounded batch (tracking the last processed ID) rather than a single migration step means it can be paused, monitored, and restarted without redoing completed work - important for a backfill that might run for hours against a genuinely large table:

```
final class BackfillCustomerEmail extends Command
{
    protected $signature =
```

```
        'orders:backfill-customer-email'
        .' {--fresh} {--dry-run} {--batch=1000}';

    public function handle(): int
    {
        $lastId = $this->option(
            'fresh'
        ) ? 0 : (int) DB::table('backfill_progress')
            ->where('name', 'customer_email')
            ->value('last_id');

        $dryRun = $this->option('dry-run');
        $orders = Order::query()
            ->whereNull('customer_email')
            ->where('id', '>', $lastId)
            ->orderBy('id')
            ->limit((int) $this->option('batch'))
            ->get(['id']);

        if ($orders->isEmpty()) {
            $this->components->info(
                'Backfill complete: no null emails left.'
            );

            return self::SUCCESS;
        }

        $nextLastId = $orders->last()->id;

        if (! $dryRun) {
            /*
             * whereNull again, so a concurrent write that
already
             * populated one of these rows isn't overwritten by a
             * stale read.
```

```
             */
            Order::query()
                ->whereIn('id', $orders->pluck('id'))
                ->whereNull('customer_email')
                ->update(
                    ['customer_email' => DB::raw('buyer_email')]
                );

            DB::table('backfill_progress')->updateOrInsert(
                ['name' => 'customer_email'],
                ['last_id' => $nextLastId],
            );
        }

        $this->components->info(
            ($dryRun ? '[dry run] ' : '')
            ."Processed {$orders->count()} rows"
            ." (last ID: {$nextLastId})."
        );

        return self::SUCCESS;
    }
}
```

Dispatched via `Schedule::command('orders:backfill-customer-email')->everyMinute()->withoutOverlapping()`, this makes progress in small, bounded increments and survives being interrupted by a deploy at any point. The cursor lives in a table rather than in the cache for a boring reason that bites people anyway: a cache is allowed to lose your key. An eviction under `maxmemory`, a `cache:clear` during a release, a Redis restart without persistence - any of those silently resets the backfill to ID 0, and the `whereNull` guard means it will quietly grind back through two hundred million already-completed rows without ever reporting that

anything went wrong.

Add `--dry-run` to every large backfill command. It should use the same query and batch-selection logic as a real run. It should report the row count and ID range without writing data. A wrong `WHERE` clause then appears as an unexpected dry-run result instead of an incorrect production update.

Also report progress after every batch. Include the row count and last processed ID. Without regular output, operators cannot distinguish a long-running backfill from a stopped one.

Downstream consumers: your schema is a public API

The phased rename protects the application. It does not protect other systems that read the database directly. Examples include a nightly analytics export, another internal service, or a business intelligence (BI) tool that queries a read replica. Dropping `buyer_email` in phase 6 breaks those consumers unless the team finds and updates them first.

The fix starts with an inventory: before any schema change on a table of nontrivial age, identify who reads it besides the application - export jobs, other services, reporting queries, scheduled dashboards. For Karbar's rename, that inventory turned up an internal reporting export that ran nightly against `orders.buyer_email` directly, owned by a different team on a different release cycle.

Two patterns handle this without forcing every downstream consumer onto the application's timeline:

- **A compatibility view.** Keep a database view that presents the old schema shape. During the dual-write window, the old column can serve

the same purpose. A consumer can continue selecting `buyer_email` after the table stops using that name. The consumer must change its `FROM` clause to use the view while the old column still exists. Drop the view only after every known consumer confirms that it is no longer needed.

- **A deprecation window with an actual notification.** Whoever owns the schema change communicates the timeline to known downstream owners the same way a public API deprecates an endpoint: an announced window, not a silent drop. This only works if the inventory step actually happened - a consumer nobody knew about can't be notified.

Both patterns above are damage control for consumers that already read columns directly. The more durable fix, when the downstream consumer is an HTTP client rather than a direct database reader, is to never let that exposure exist in the first place: don't return `Model::toJson()` or a bare `$order->toArray()` from a controller, and don't let a resource's field names default to mirroring column names. A Laravel API Resource (or a `league/fractal` transformer, on a project that predates or doesn't use Resources) sits between the two and makes the wire format an explicit, independently versioned contract:

```
final class OrderResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            // Wire format is frozen; the column behind it is
not.
            'buyer_email' => $this->customer_email,
            'status' => $this->status,
            'total_cents' => $this->total_cents,
        ];
    }
}
```

With that boundary in place, the entire six-phase rename in this chapter can happen without a single HTTP API consumer noticing - the resource is the only place that has to change, and it changes the other way round from what you'd expect: it maps the new column onto the response field the API already promised, instead of the API dictating what the column is allowed to be called.

For the rename, the old `buyer_email` column stayed in place, populated by the dual-write step, until the reporting team confirmed their nightly export had been updated to the new column name - on their schedule, not the application's. Only then did phase 6 ship. Had the export needed more time than the dual-write window allowed, a compatibility view would have covered the gap - shipped during the deprecation window rather than alongside the drop, since a view that appears on the same day the column vanishes has nobody pointed at it yet:

```
-- Ships during the deprecation window, well before phase 6's
drop, so
-- consumers have time to repoint FROM orders at
orders_legacy_shape and
-- keep buyer_email in their SELECT list afterwards.
CREATE VIEW orders_legacy_shape AS
    SELECT id, customer_id, customer_email AS buyer_email,
status, total_cents
    FROM orders;
```

A schema-change checklist

```
- [ ] Tested the specific operation (not just "a migration")
against a production-sized copy of the table
- [ ] Confirmed whether the operation takes a blocking lock on
the current engine/version
- [ ] Used a non-blocking index build (CONCURRENTLY /
ALGORITHM=INPLACE) for large tables
- [ ] Renames/type changes are split into add -> dual-write ->
backfill -> migrate reads -> drop phases
- [ ] Backfills run in bounded batches with a pause, not as one
statement
- [ ] Inventoried downstream consumers (exports, other services,
BI tools) that read this table directly
- [ ] HTTP API responses go through a Resource/transformer, not a
bare model - column renames don't reach API clients
- [ ] Downstream consumers notified or bridged with a
compatibility view before the old shape is dropped
- [ ] The "drop" phase is the last, separately reviewed step -
never bundled with the rest
```

Chapter recap

- A migration's cost scales with table size and the specific engine/operation, not with how simple the change looks in code - always test DDL against production-sized data.
- Use non-blocking index builds (`CREATE INDEX CONCURRENTLY` on PostgreSQL, `ALGORITHM=INPLACE, LOCK=NONE` on MySQL) on any table large enough that a lock would be noticed.
- Never rename or retype a column in one step - expand it into add, dual-write, backfill, migrate reads, stop dual-write, and drop, each independently deployable and reversible until the last phase.
- Backfill large tables in bounded, pausable batches (`chunkById` plus a

small delay), not a single sweeping `UPDATE`, to protect replication and live traffic.

- Your schema is effectively a public API the moment anything outside your own deploy - an export, another service, a BI tool - reads it directly. Inventory those consumers before assuming a change is safe.
- For consumers that go through HTTP rather than the database directly, a Laravel API Resource (or `league/fractal`) makes the response shape an explicit contract instead of a mirror of column names - the safest fix is the one that prevents the exposure from existing at all.
- Bridge downstream consumers with a compatibility view or an announced deprecation window; let the slowest consumer's timeline, not your migration's convenience, decide when the old shape can actually be dropped.

Chapter 10 - Caching Strategies

The outage the database didn't cause

Drishti, a fictional short-video platform, serves a ranked "trending" feed. The ranking uses view counts, likes, and recency. Computing it requires several database tables and takes a few hundred milliseconds.

The team adds a cache. The application computes the feed once and stores it in Redis with a five-minute time to live (TTL). Requests use the cached value until it expires. Response time drops from 300ms to 2ms.

Four months later, the feed fails for ninety seconds during peak traffic. The database appears to be the cause, but normal database load was not the problem.

The cache key expired at exactly 12:00:00. In the same second, eleven thousand concurrent requests found no cached value. Every request started the same expensive aggregation. The hard TTL changed steady read traffic into a synchronized cache stampede. This sudden load overwhelmed the database.

Careless caching can increase peak load instead of reducing it. This chapter explains how to prevent that failure while keeping the speed and load reduction that caching should provide.

Two limiter algorithms, and why the difference matters

Laravel provides one of the two algorithms below. The `RateLimiter` facade and `throttle` middleware both count requests in cache-based fixed windows. Confirm the algorithm before deciding that an endpoint has enough protection.

Fixed window counts requests in discrete buckets (e.g. "requests this calendar minute"). It's cheap - one counter, one TTL - but it has a boundary problem: a client can send its full limit at 11:59:59 and its full limit again at 12:00:01, getting two limits' worth of traffic in two seconds.

Token bucket (and its close cousin, the sliding-window log) models a bucket that refills at a steady rate and is drained by each request. It smooths out the boundary problem because there's no reset instant - the bucket's state is continuous. It costs slightly more to compute (you need the last-refill timestamp, not just a count) but it's a much closer match to "how many requests can this client sustainably send," which is usually the thing you actually care about.

	Fixed window	**Token bucket / sliding window**
Storage cost	One counter + TTL per key	Timestamp + count per key
Boundary burst	Up to 2x limit across a window edge	Bounded by bucket size, not window edge

	Fixed window	Token bucket / sliding window
Good for	Coarse, cheap limits (e.g. "1000 req/hour per API key")	Limits where burst behavior matters (login, payment attempts)
Laravel primitive	`RateLimiter::attempt()` with default cache-based windows	Custom atomic bucket in Redis or an edge-provider limiter

For most application throttles, the fixed window's imprecision is acceptable. A small burst on a reporting endpoint may cause no harm.

Security-sensitive actions need stricter control. Examples include login, password reset, payment attempts, and one-time password (OTP) verification. For these actions, a window-boundary burst is a security gap. Use a token bucket in Redis or at the edge. Laravel's standard limiter does not provide a token-bucket mode.

Choosing the key: per-user, per-IP, per-action, or a combination

A rate limit depends on its key. Many simple implementations choose the wrong key.

- **Per-IP** limits stop a single machine from hammering an endpoint, but are close to useless against a distributed botnet, and actively harmful behind carrier-grade NAT or corporate proxies, where thousands of

real users share one visible IP.
- **Per-user (or per-account)** limits are precise but only apply once you know who the user is - they don't help on an unauthenticated login attempt where the "user" is exactly what's being guessed.
- **Per-action** limits (e.g. "5 password reset emails per hour, regardless of who's asking") protect a shared downstream resource (an email provider, an SMS gateway) from being exhausted by any single actor.
- **Composite keys** - IP *and* the attempted username, or device fingerprint *and* account - are usually where the real protection lives, because they let you set a tight limit on the narrow combination ("this IP trying this specific username") without punishing an entire shared IP range for one bad actor.

The credential-stuffing scenario shows why one key is not enough. The botnet defeats per-IP limits by using many addresses. A per-username limit alone still allows one attempt against every account.

Apply several limits with different keys at the same time. Each limit is inexpensive to enforce. An attacker must defeat all of them.

```
/*
 * A layered login throttle: IP-wide, then IP+username, then
 * account-wide.
 */
public function attempt(Request $request): RedirectResponse
{
    $ip = $request->ip();
    $username = (string) $request->input('email');

    $limits = [
        // this IP, any account, per minute
        "login-ip:{$ip}" => 30,
```

```
        // this IP against this account
        "login-ip-user:{$ip}:{$username}" => 5,
        // this account, from anywhere
        "login-user:{$username}" => 10,
    ];

    foreach ($limits as $key => $maxAttempts) {
        if (RateLimiter::tooManyAttempts($key, $maxAttempts)) {
            $seconds = RateLimiter::availableIn($key);

            throw ValidationException::withMessages([
                'email' =>
                    "Too many attempts. Retry in {$seconds}s.",
            ])->status(429);
        }
    }

    if (! Auth::attempt($request->only('email', 'password'))) {
        foreach (array_keys($limits) as $key) {
            /*
             * The first failed attempt must create the fixed
             * window's TTL.
             */
            RateLimiter::hit($key, decaySeconds: 60);
        }

        throw ValidationException::withMessages([
            'email' =>
                'These credentials do not match our records.',
        ]);
    }

    /*
     * Successful logins do not increment any of the failure
     * counters.
```

```
        */
        return redirect()->intended('/dashboard');
    }
```

Hitting the limiter only on *failed* attempts, rather than on every request, is what keeps this from punishing legitimate traffic. A limiter that counts successes too will eventually lock out the user whose only crime is logging in a lot.

Bot and risk scoring: don't let a vendor outage become your outage

Many teams layer a third-party risk-scoring or bot-detection service (device fingerprinting, behavioral scoring, a managed CAPTCHA) on top of application-level rate limits. These are valuable, but they introduce a dependency that can fail independently of your own infrastructure - and a naive integration turns "the risk vendor is slow" into "nobody can log in."

Decide explicitly what happens when the risk check fails or times out. Do not let the exception produce a `500` response. Two valid approaches exist. The correct approach depends on what the endpoint protects:

Failure mode	Fail closed (block)	Fail open (allow)
Behavior on vendor timeout	Treat as high-risk, block or challenge	Treat as unscored, let the request through

Failure mode	Fail closed (block)	Fail open (allow)
Right for	High-value, low-volume actions (large withdrawals, account recovery)	High-volume, low-marginal-risk actions (viewing a page, browsing a catalog)
Failure mode if wrong	Vendor blip becomes a full outage for real users	A vendor outage becomes a temporary abuse window
Mitigation	Short timeout (100-300ms) + circuit breaker so a slow vendor degrades to "unscored" quickly	Alert loudly on sustained fail-open so someone investigates rather than it becoming permanent

For Adda's login endpoint, always apply the self-hosted application rate limits. Treat the vendor risk score as an *additional* signal. A high score can require a CAPTCHA, but the base throttle must not depend on the vendor.

During a vendor outage, bot detection becomes less precise. Login remains available.

In code, that means a tight timeout on the vendor call and an explicit fail-open branch, rather than letting the exception propagate:

```
public function scoreLogin(string $ip, string $email): ?RiskScore
{
    try {
        $response = $this->http
```

```
            // 250ms: fail fast, don't hold up the login request
            ->timeout(0.25)
            ->post('https://risk-vendor.example.com/v1/score', [
                'ip' => $ip,
                'email' => $email,
            ]);

        if ($response->failed()) {
            /*
             * A 4xx/5xx arrives here, not in the catch block
             * below.
             */
            Log::critical(
                'Risk vendor returned an unsuccessful response',
                [
                'status' => $response->status(),
            ]);

            return null;
        }

        return RiskScore::fromArray($response->json());
    } catch (ConnectionException $e) {
        /*
         * Fail open: an unscored login still passes the
         * application-level throttle above, it only skips the
         * extra CAPTCHA/challenge step.
         */
        Log::critical(
            'Risk vendor unavailable, failing open',
            ['exception' => $e->getMessage()]
        );

        return null; // caller treats null as "unscored"
    }
```

```
    }
```

The loud `Log::critical` call is what keeps a fail-open decision visible instead of silently permanent.

DDoS protection lives above Laravel

The credential-stuffing attack is application abuse. It uses many valid HTTP requests that resemble real login attempts. A distributed denial-of-service (DDoS) attack is different. It tries to exhaust bandwidth, file descriptors, or worker capacity before Laravel can process the requests.

Laravel's `RateLimiter` cannot help when requests never reach a PHP worker. Application abuse limits and DDoS protection must operate at different layers.

The layer that can absorb a volumetric flood is the one in front of your origin: a CDN or DDoS-aware edge (Cloudflare, AWS Shield / WAF, Fastly, and their peers), with connection limits and SYN cookies at the load balancer as a second line. That edge is where you put IP reputation, geographic blocks you actually intend to enforce, managed bot challenges, and traffic shaping that can drop junk without waking a Laravel container. Application rate limits still matter for the abuse cases this chapter covers - they are just not the DDoS plan.

Layer	What it can stop	What it cannot
Edge / CDN / WAF	Volumetric floods, obvious botnets, TLS exhaustion, cheap challenge-before-origin	Business-logic abuse that looks like a real user (credential stuffing with residential IPs)
Load balancer / reverse proxy	Connection storms, slowloris-style holds, per-IP connection caps	Anything that already looks like a healthy HTTP request
Laravel `RateLimiter` / `throttle`	Per-key abuse once a request reaches the app	Floods that saturate the link or the worker pool before PHP runs

For Adda, the practical split after the stuffing incident was: keep the layered login throttles in the app (they still catch the "looks like a user" case), and put volumetric protection at the edge so a packet flood never becomes a Horizon backlog. Two other habits make the origin cheaper to defend when something does leak through:

- **Fail expensive work late.** Don't run bcrypt, Eloquent, or a risk-vendor call before the cheapest checks (routing, auth middleware, a cache hit on a known-bad IP). Every millisecond of origin work during a flood is capacity an attacker can buy more cheaply than you can.
- **Keep health and static paths cheap and separate.** A `/up` or health probe that still boots the full application, or a homepage that always misses cache and hits the database, turns "protect the login endpoint"

> into "the whole fleet is busy answering probes and HTML." Edge caching for anonymous GETs, and a health check that doesn't touch Redis or MySQL, shrink the surface an attack can usefully burn.

What you should not do is invent an in-app "DDoS mode" that flips every limit to zero when CPU is high. That couples abuse policy to capacity noise, locks out real users during a legitimate spike (the flash-sale case Chapter 16 plans for), and still does nothing if the attack never reaches PHP. Edge absorption first; application throttles for the requests that deserve application judgment.

Backpressure: what a limit does after it triggers

A rate limit isn't just "reject the request." What you return, and what you do internally when a limit is being approached, is its own design surface:

- **Return `429 Too Many Requests`**, not a generic `403` or `500` - well-behaved clients (and API consumers) know to back off on a 429 specifically, and a `Retry-After` header lets them do it without guessing.

Concretely, that's `RateLimiter::availableIn()` feeding straight into the response:

```
public function login(Request $request): JsonResponse
{
    $key = "login-ip:{$request->ip()}";

    if (RateLimiter::tooManyAttempts($key, 30)) {
        $retryAfter = RateLimiter::availableIn($key);

        return response()->json(
            [
                'message' =>
                    "Too many attempts. Retry in {$retryAfter}s."
            ],
            429,
        )->header('Retry-After', $retryAfter);
    }

    // ...
}
```

- **Shed load progressively, not as a cliff.** If an upstream dependency (a payment gateway, a third-party API) is itself struggling, consider tightening your own limits proactively rather than waiting for it to start timing out request-by-request - this is the same backpressure idea covered for queues in Chapter 12, applied at the edge instead of the worker layer.
- **Distinguish abuse throttling from capacity throttling.** A login rate limit exists to stop abuse and should feel deliberately strict. A general API rate limit protecting your own database from being overwhelmed is a capacity control, and Chapter 16's load-testing work is what tells you where that number should actually sit - guessing it produces either a limit so loose it doesn't protect anything, or so tight it throttles normal peak traffic.

Chapter recap

- Rate limiting is a layered decision, not a single mechanism: pick an algorithm (fixed window for cheap coarse limits, token bucket for anything where burst behavior matters), a key (IP, user, action, or a composite), and a response (reject, challenge, or degrade).
- Composite keys (IP + username, device + account) catch attacks that defeat any single-axis limit, including distributed credential stuffing that no per-IP limit alone can stop.
- Increment attempt counters on failure, not on every request, or you'll rate-limit your legitimate power users.
- Third-party risk/bot signals are an enhancement, not a foundation - decide explicitly whether each check fails open or closed, and never let a vendor timeout become your outage.
- DDoS is an edge problem, not a `RateLimiter` problem: absorb volumetric floods at the CDN/WAF/load balancer, keep origin work cheap for what leaks through, and reserve application throttles for abuse that looks like a real request.
- Return `429` with `Retry-After`, and treat abuse throttling (strict, security-motivated) and capacity throttling (informed by load testing, see Chapter 16) as two different tuning problems with two different failure costs.

Chapter 16 - Load Testing & Capacity Planning

The load test that passed, and the launch that didn't

Karbar, a fictional retail platform, is preparing for its largest marketing event of the year. The flash sale is expected to produce ten times normal traffic for two hours.

The team load-tests the `/checkout` endpoint with one thousand concurrent requests. Latency stays below 200ms, and the error rate stays near zero. The team marks the test as passed.

The site still fails during the sale. `/checkout` alone is not the cause. Real shoppers browse products, update carts, leave, return, and sometimes complete checkout. These actions happen together.

The single-endpoint test did not reproduce this traffic mix. It did not test the database connection pool under concurrent reads and writes. It also missed the actual capacity limit: the third-party payment gateway throttles Karbar after a request threshold that the team did not check.

The load test produced valid results for the scenario it tested. The scenario was wrong. It covered one endpoint and one traffic shape. The only pass condition was that the endpoint did not return a `500`.

This chapter explains how to model expected traffic and define success before testing.

timezone for calendar math. This chapter explains the boundary cases created by rules such as "day," "week," and "Monday at midnight."

Instants and calendar days are different types

An *instant* is one exact point on the timeline. It can be stored once and compared everywhere. A *calendar day* is a civil label that depends on a timezone. "Tuesday, March 10" in Vancouver contains a different set of instants from "Tuesday, March 10" in UTC. Mixing these two types causes the main failure.

Question	Use
When did this row get written?	UTC instant (`timestampTz`, `now()` in UTC)
Is this user still inside a 24-hour redeem window?	Duration from a UTC instant (`subHours(24)`)
Did they act on consecutive *local* days?	Convert to business TZ, then `toDateString()`
Does "this week" mean Mon-Sun for the campaign?	Week boundaries in the business TZ
When should the Monday payout cron run?	Scheduler `->timezone( ... )` matching that calendar

Laravel's `config('app.timezone')` should almost always be `UTC`. That makes `now()`, Eloquent timestamps, and queue `available_at` values mean

the same thing on every server and in every region. Business calendar semantics belong in application code as an explicit constant - not in `APP_TIMEZONE`, and not as a per-row column unless you genuinely need per-user timezones (wallets with local spending days, Chapter 19's `spendingDay`, are that case; a single-company campaign usually is not).

```
final class CampaignConfig
{
    /*
     * Calendar days, weeks, and payout schedules for this
     * product. DB timestamps stay UTC instants; convert to
     * this zone only when asking civil-date questions.
     */
    public const BUSINESS_TIMEZONE = 'America/Vancouver';
}
```

One constant is safer than a `timezone` column that some rows might omit. It also prevents inconsistent literals such as `America/Vancouver`, `America/Los_Angeles`, and `PST` across jobs.

Convert at the boundary, not in the middle

When an event arrives - a watch, a purchase, a ticket grant - keep the instant in UTC, and derive the civil day only where the product rule needs it:

```
public function recordActiveDay(
    int $userId,
    CarbonImmutable $activeAt,
): void {
    $activeDate = $activeAt
        ->timezone(CampaignConfig::BUSINESS_TIMEZONE)
        ->startOfDay();
    $today = $activeDate->toDateString();

    $progress = StreakProgress::query()
        ->lockForUpdate()
        ->firstOrCreate(
            [
                'user_id' => $userId,
                'active_date' => $today,
            ],
        );

    /*
     * ... streak increment using $today / yesterday
     * in the same business timezone
     */
}
```

Call `->timezone(BUSINESS_TIMEZONE)` *before* `startOfDay()` or `toDateString()`. If you call `startOfDay()` in UTC and then convert to Pacific time, you calculate the wrong day. Queries require the reverse conversion. For "everything since one week ago at Pacific midnight," calculate the boundary in the business timezone. Then convert it to UTC for the SQL comparison:

```
$weekAgoPacificMidnightUtc = CarbonImmutable::now(
    CampaignConfig::BUSINESS_TIMEZONE,
)
    ->subWeek()
    ->startOfDay()
    ->utc();

Campaign::query()
    ->where('ends_at', '>=', $weekAgoPacificMidnightUtc)
    ->get();
```

That pattern - civil math in the business zone, storage and filters in UTC - is the same idea Chapter 19 uses when it freezes `spendingDay` at decision time so replay never re-asks "what is today?"

Schedulers must declare the timezone they mean

Laravel's scheduler inherits `app.timezone`. If that is UTC, then `->dailyAt('00:05')` means 00:05 UTC, not "just after midnight for the Pacific campaign." The failure mode from the opening incident is usually quieter than a wrong app timezone: one job uses `->timezone('America/Vancouver')`, a sibling job forgets and runs in UTC, and both claim to be "the Monday payout."

```
$schedule->job(new DisburseStreakRewards)
    ->weekly()
    ->mondays()
    ->at('00:05')
    ->timezone(CampaignConfig::BUSINESS_TIMEZONE)
    ->withoutOverlapping()
    ->onOneServer();

$schedule->command('campaign:sync-all-items')
    ->dailyAt('06:00')
    ->timezone('UTC')
    ->withoutOverlapping()
    ->onOneServer();
```

Declare the timezone even when it is UTC. A codebase can contain Vancouver payouts, New York compliance jobs, and Los Angeles digests. Each schedule entry must name its timezone next to its clock time, and that timezone must match the product rule. An entry without `->timezone()` uses the current `app.timezone`. It is not independent of timezones, so it is unsafe for wall-clock work.

Uniqueness keys must use the same clock as the schedule

`ShouldBeUnique` and cache locks are timezone-sensitive when the key embeds a date or hour. A streak reward job that must run once per Pacific calendar day needs a Pacific day in `uniqueId()`. A missing-ticket generator that may safely run once per UTC hour needs a UTC hour. Using the wrong one either blocks a legitimate second run or allows a duplicate:

```
public function uniqueId(): string
{
    /*
     * Pacific calendar day - aligned with the Monday
     * Vancouver schedule, not with UTC midnight.
     */
    return $this->campaign->slug.'::'.now(
        CampaignConfig::BUSINESS_TIMEZONE,
    )->format('Y-m-d');
}
```

```
public function uniqueId(): string
{
    // Hourly reconciliation: UTC hour is the right grain.
    return $this->campaign->slug.'::'.now()
        ->format('Y-m-d-H');
}
```

The unique key and schedule must use the same definition of midnight. If they differ, work can be skipped or payouts can run twice. The failure appears only near timezone boundaries, which makes it difficult to reproduce.

"Day" sometimes means 24 hours

Not every product "day" is a calendar day. A redeem cap of "once per day" often means "no more than once in any rolling 24-hour window," which is a duration from a UTC instant:

```
$recent = Redeem::query()
    ->where('user_id', $userId)
    ->where('created_at', '>=', now()->subHours(24))
    ->exists();
```

A weekly purchase limit that uses `startOfWeek()` in UTC while marketing promises "this Pacific week" is a quieter cousin of the streak bug. Decide deliberately:

- **Rolling duration** (`subHours(24)`, `subDays(7)`) when the rule is elapsed time.
- **Civil calendar** (business-TZ `startOfDay` / `startOfWeek`) when the rule is a named day or week the user would recognize on a wall calendar.

Write the choice down next to the constant. The incident is almost always someone using the other interpretation without noticing.

Don't write time as a magic number

Durations have the same clarity problem as timezones. A bare `300` in a cache call does not identify its unit. Laravel treats integer TTLs as seconds, so code written under a minutes assumption can fail silently. Use a helper that names the unit.

```
use function Illuminate\Support\{
    minutes,
    seconds,
    hours,
    days,
};

/*
 * Readable at the call site - five minutes, not "300 of
 * something." These return a CarbonInterval the cache
 * layer accepts directly.
 */
Cache::put('trending', $feed, minutes(5));
Cache::remember(
    'exchange-rate',
    seconds(30),
    fn () => $this->fetchRate(),
);
Cache::put('sitemap', $xml, hours(6));
Cache::put('annual-report', $pdf, days(1));
```

`now()->addMinutes(5)` (or `now()->plus(minutes: 5)`) is the same idea when you need an absolute expiry instead of an interval - Chapter 10's cache examples use that form. What to avoid is the unadorned integer and the arithmetic that pretends to document itself:

```
// Opaque: seconds? minutes? a typo for 30?
Cache::put('trending', $feed, 300);

// Still a magic number, just wearing a disguise.
Cache::put('trending', $feed, 5 * 60);
```

If a TTL is a product decision reused in more than one place, lift it to a named constant (`private const TRENDING_TTL = ...` backed by `minutes(5)`), the same way Chapter 17 names `TTL_MINUTES` next to the partner idempotency cache. The goal is that a reviewer can read the call site and know the unit without doing mental arithmetic or checking the framework docs for which epoch the integer means this year.

Admin panels lie in the default timezone

Operators use the admin interface to investigate production. If it displays `created_at` in UTC while the team uses Pacific time, an operator can approve the wrong payout day or reopen the wrong dispute. A Monday job can also appear to have run on Sunday evening.

Set a display default in the business timezone, and make sure date columns actually go through the datetime formatter - a bare text column ignores the panel timezone entirely:

```
/*
 * Panel default: America/Vancouver. Columns still need
 * ->dateTime() (or equivalent) so the timezone applies;
 * a plain TextColumn shows the raw UTC string.
 */
Table::configureUsing(function (Table $table): void {
    $table->timezone(
        CampaignConfig::BUSINESS_TIMEZONE,
    );
});
```

Label ambiguous fields in the UI ("Starts at (Pacific)") when compliance or finance will read them. For user-facing legal timestamps where UTC is the contract, label UTC explicitly instead of "helpfully" converting.

DST, ambiguous hours, and tests that freeze the wrong clock

`America/Vancouver` (and any IANA zone with daylight saving) has days that are 23 or 25 hours long. Local times between 2:00 and 3:00 may not exist, or may exist twice, on transition nights. Prefer:

- IANA names (`America/Vancouver`), never fixed offsets (`UTC-8`) or abbreviations (`PST` / `PDT`) as the source of truth. A fixed offset is wrong half the year wherever daylight saving applies - `UTC-8` is Pacific *Standard* Time, so every civil midnight you compute with it during PDT is an hour off. Abbreviations are worse: they name a single offset (`PST` vs `PDT`), they collide across regions (`CST` is Central in North America and China Standard Time elsewhere), and they force every call site to know which seasonal label is "current" instead of letting the

zone database apply the transition rules.
- Civil boundaries via `startOfDay()` / `startOfWeek()` in that zone, not `setTime(0, 0)` on a UTC instant you hope is midnight.
- Storing the *result* of calendar decisions when they must survive replay (Chapter 19's frozen `spendingDay`), instead of recomputing "today" later.

Tests are where these bugs either get caught or get permanently skipped. Pin time in the zone the rule uses:

```
public function test_streak_counts_pacific_days(): void
{
    CarbonImmutable::setTestNow(
        CarbonImmutable::parse(
            '2026-03-10 23:30:00',
            CampaignConfig::BUSINESS_TIMEZONE,
        ),
    );

    /*
     * act in Pacific evening...
     * assert active_date === '2026-03-10', not 03-11
     */
}
```

Also test the UTC representation of the same instant: `06:30` on the next UTC day during Pacific Daylight Time (PDT). This test detects a future refactor that removes `->timezone( ... )`. Add dedicated tests for these boundaries:

1. One second before and after Pacific midnight.
2. The spring-forward gap.

3. The fall-back overlap.
4. The first and last partial weeks of a campaign that does not start on Monday.

A short checklist before shipping calendar logic

1. `app.timezone` is UTC; business calendar TZ is a named constant.
2. Every `startOfDay` / `startOfWeek` / `toDateString` for product rules runs after converting to that constant.
3. Every scheduler entry that cares about wall-clock time calls `->timezone( ... )` explicitly.
4. Unique locks and idempotency keys embed dates or hours in the *same* zone as the schedule or product rule they protect.
5. Rolling windows use durations; civil windows use the business zone - never both for the same rule.
6. Cache TTLs and other durations use `minutes()` / `seconds()` / `hours()` / `days()` (or `now()->addMinutes( ... )`), not bare integers like `300`.
7. Admin date columns format through the panel timezone; labels say which zone the human is looking at.
8. Tests freeze time in the business zone and assert the civil date, not only the UTC timestamp.

Chapter recap

- Timezone bugs are silent correctness failures: wrong streak days, wrong week boundaries, wrong cron walls, wrong admin timestamps - usually without an exception.
- Keep `app.timezone` at UTC for instants; put civil-calendar semantics in an explicit business-timezone constant and convert only at the

boundary where the product asks a calendar question.

- Schedulers and uniqueness keys must name the same timezone as the product rule they implement; "default app timezone" is not a safe stand-in for "Monday midnight Pacific."
- Decide whether "day" means a rolling duration or a civil calendar day, and encode that choice once - mixing the two is how redeem caps and streak counters drift apart.
- Write durations with unit-named helpers (`minutes(5)`, `seconds(30)`, `now()->addMinutes(5)`), not magic numbers - bare `300` hides the unit and has already bitten teams when cache TTLs changed from minutes to seconds.
- Admin UIs need the business timezone *and* actual datetime formatting; otherwise operators debug the wrong clock.
- Prefer IANA zones, freeze civil decisions that must survive replay, and write tests around Pacific midnight and DST transitions - those are the hours production will find if you don't.

Part 4 - High Availability & Resilience

This part explains how to keep a system available, or provide reduced service safely, when a dependency fails or a data center loses power. It also explains how to disable a risky release in seconds instead of minutes.

This is a sample from "Laravel After Deploy: Architecture, Performance, and Operations at Scale".